

sdpa-dd-omp: threading a double-double SDP solver, and making its threaded results reproducible

sdpa-dd-omp — technical record

2026-08-25

Built from `doc/technical.tex`, SHA-256
f868f0b5785cb94c5a644a8d28c3546a87e094151351d3d4f12c623850c668a8.

Abstract

sdpa-dd solves semidefinite programs in double-double arithmetic — roughly 32 significant decimal digits, carried as an unevaluated sum of two IEEE doubles. It builds with OpenMP, but threading it as shipped is not merely unprofitable, it is unsound: on small problems more cores make it monotonically slower ($24.5\times$ slower on 24 cores than on one, on a user’s $m=16$ input), and on any problem its threaded runs are not reproducible.

This document records what was changed and why. Five things are worth a reader’s attention. First, the dominant cost is the Schur-complement build and factorisation, and on large sparse problems upstream threads *neither* the sparse factorisation nor its assembly, so twenty-four cores buy it 4.9%. Second, threading a multiprecision solver is mostly a *reproducibility* problem: this fork’s assembled Schur complement and its Cholesky factor are bit-identical at any thread count *by construction*, and that is asserted against the structures themselves rather than against printed digits. Third, one of this fork’s own released versions shipped a data race, and the account of how it was missed is more useful than the fix. Fourth, the rule choosing between a dense and a sparse factorisation of the Schur complement was mis-tuned in a way a retune would not have found: one of its four gates is not a heuristic at all but a provable consequence of another. Fifth, on **aarch64** the compiler’s default floating-point contraction silently changes double-double arithmetic, so results differed between platforms until contraction was pinned off.

Contents

1	The solver, and where the time goes	3
2	Kernel and threshold defects in the inherited code	3
2.1	The lost zero-skip in <code>Rgemm</code>	3
2.2	The bundled QD builds without hardware FMA on aarch64	3
2.3	OpenMP regions entered regardless of problem size	4
2.4	Timers that reported worker-seconds as elapsed time	5
3	Reading high-precision input	5
4	Floating-point contraction, and why this fork pins it off	5
4.1	The mechanism	5
4.2	The fix, and its cost	5

5	What was threaded, and what it costs correctness	6
5.1	The B constraint loop	6
5.2	The sparse Schur-complement Cholesky	6
5.3	The sparse Schur-complement assembly	6
5.4	Cholesky panel kernels and the X/Z inverse-Cholesky triangulars	6
5.5	The triangular solves: forward yes, backward no	6
6	A data race in the threaded assembly, and its fix	7
6.1	What shipped	7
6.2	Why nothing caught it	7
6.3	The fix	8
6.4	The regression fixture, and why its first version was worthless	8
7	The factorisation choice	8
7.1	Four gates	8
7.2	One constant doing two unrelated jobs	9
7.3	Gate 3 is not a heuristic	9
7.4	What the old value was, and why it was wrong	9
7.5	Who this affects	9
7.6	Measured effect	10
7.7	What this evidence does not establish, and where the rest of it comes from . . .	10
7.8	Not “sparse always wins”	10
7.9	What the change costs, stated plainly	10
8	Reproducibility: established, not asserted	11
8.1	What is guaranteed	11
8.2	Why the printed solution is the wrong thing to compare	11
8.3	The oracles	11
8.4	Negative controls, and why they are not optional	12
8.5	Route fixtures	12
9	Environment variables	12
9.1	Choosing how the Schur complement is stored and factored	13
9.2	Threading the sparse Cholesky	13
9.3	Threading the sparse B assembly	14
9.4	Profiling and census	14
10	Exit status	14
11	What is not established	15
12	Sources	15

1 The solver, and where the time goes

At each interior-point iteration the solver forms the $m \times m$ Schur complement B (the “bMat”), factorises it, and performs a forward and a backward triangular solve against the factor. In double-double arithmetic every scalar operation is an inlined sequence of some ten to twenty IEEE operations with strict ordering requirements, so the constant factors a double-precision solver hides become the whole story.

Two regimes matter, and they have different bottlenecks.

Small and medium problems ($m \lesssim 10^3$). B is dense, and the cost is the constraint loop that builds it plus a dense Cholesky. This is the regime SDPLIB covers, and it is the regime upstream’s own threading was written for.

Large sparse problems ($m \sim 10^3\text{--}10^4$). B has a sparsity structure worth exploiting, and the solver can store and factor it sparsely. Here the cost splits between the *sparse assembly* — scattering constraint-pair contributions into the stored nonzeros — and the *sparse Cholesky*. **Upstream threads neither.** This is why twenty-four cores buy upstream 4.9% on dE4 ($m=7401$): the problem routes sparse, and both of the regions that dominate it are serial code. It is also why the SDPLIB tables understate this fork: every SDPLIB problem takes a dense B and never enters the path where the difference lives.

A methodological note, recorded because the discarded hypothesis is part of the result: the first optimisation proposed for the sibling `sdpa-gmp` port was a per-block parallel Cholesky over the SDP blocks. Profiling assigned 66–80% of runtime to the B build, and the proposed region did not appear in the profile at all. It was abandoned on the evidence.

2 Kernel and threshold defects in the inherited code

2.1 The lost zero-skip in Rgemm

Netlib `dgemm` guards its inner update with a test for a zero multiplier. In `mplapack` 2.0.1 that guard was dropped from the OpenMP variants. In double precision the guard is a minor optimisation; in double-double it is not, because “multiply by zero” still pays a full multiprecision call. Restoring it in `mplapack/Rgemm_NN_omp.cpp` and `Rgemm_NT_omp.cpp` recovers up to $2\times$ on the serial path.

2.2 The bundled QD builds without hardware FMA on aarch64

QD’s own `configure` runs its FMA probe only under `case $host in powerpc*-*-*)`. The default `--enable-fma=auto` therefore resolves silently to *none* everywhere else: `QD_FMA/QD_FMS` stay undefined, and `two_prod` — the most-executed primitive in the whole solver — compiles the Dekker two-way split instead of $p = ab$; `err = fma(a, b, -p)`.

On Apple Silicon that is **72 instructions against 8** (`dd_real::operator*`: 71 against 13), the difference being `fmul+fnmsub` against a sequence containing two conditional branches for `split()`’s 2^{96} overflow guard. Enabling it moves the solver’s main loop by $1.59\times$ **in aggregate** at one thread — per problem 1.21–1.63 $\times$ (`truss6` 1.21, `arch0` 1.38, `mcp250-1` 1.53, `gpp250-1` 1.61, `control7` 1.62, `theta3` 1.63) — against a repeat spread of 0.1–2.8%, so the smallest ratio exceeds the largest spread by about $4\times$ and no row is marginal.

Bit-identical, and measured rather than assumed. Sixteen SDPLIB problems were hashed over the complete solution section: reference-versus-candidate at 8 threads 0/16, candidate 1 thread versus 8 threads 0/16, reference-versus-candidate at 1 thread 0/16, and both arms under `-ffp-contract=on` 0/16. Iteration counts were identical across all 96 runs. The comparison is proven *sensitive* rather than merely quiet: perturbing `lambdaStar` by one part in 10^{13} with the same binary moved 17 fields across 6 problems. A scalar cross-check over $\sim 106\text{M}$ random pairs found the product and the error term agreeing exactly across the solver band, the normal band, `two_sqr`, near-overflow and huge-times-tiny; they diverge only for deep subnormals, where the exact error term is below `DBL_MIN` and unrepresentable.

Strictly more correct, too. Near `DBL_MAX` the Dekker split overflows and returns `inf/nan` for a finite product — $a = 10^{-300}$, $b = \text{DBL_MAX}$ gives $\text{err} = \infty$ under the split and 9.84×10^{-9} under FMA. That is a latent defect in the non-FMA builds, removed here on aarch64.

The aarch64 gate is deliberate, not an oversight. On aarch64 `fmadd` is baseline ISA, so `__builtin_fma` is a single instruction. On x86-64 *without* `-mfma`, GCC lowers `__builtin_fma` to a libm *call*, which is far slower than the split — and this project builds generic x86-64 on purpose (no `-march`, no `-mfma`) so binaries stay bit-identical across AMD and Intel nodes. On every non-aarch64 host the argument handed to QD’s `configure` is unchanged, and x86-64 was verified untouched at seven levels, including 74/74 objects identical after stripping, `.text` byte-identical, and all 60 published state-hash cells regenerated and matched. `--enable-qd-fma=auto|gnu|no` overrides the host gate, and exists so that “the same build with QD’s explicit FMA turned off” is configurable without hand-editing — hand-editing does *not* invalidate `external/qd`’s configure stamp, and would silently reuse the previous library.

One inherited trap was fixed alongside it: `work/.qd_configure.done` depended only on the patch stamp, so an existing build tree kept a stamp made with the old arguments and silently produced a non-FMA library. Anyone benchmarking in an existing tree would have measured the reference while believing they measured the candidate.

Known gap, deliberately not closed here. `install.sh` short-circuits to a system QD on Linux when `/usr/include/qd/qd_real.h` exists, so an aarch64 Linux box with a distribution `libqd-dev` never reaches this flag. Fixing that would also change which QD x86-64 Linux picks up, and so needs its own decision.

2.3 OpenMP regions entered regardless of problem size

Upstream enters its parallel regions without asking whether there is enough work to repay the fork/join. It contains a threshold mechanism, but the mechanism is disabled behind `if (0)`, and it cannot simply be switched on: the `_ref` bodies it dispatches to are absent and the tree would not link.

The consequence is not a small inefficiency. On a user’s `max_custom_scaled.dat-s` ($m=16$), upstream degrades *monotonically* at every thread count, ending $24.5\times$ slower on 24 cores than on one. On SDPLIB `control1` it is $11\times$ slower at 24 threads than at 1; on `truss5`, $3.3\times$ slower. This fork gates entry on work, matrix width and nesting depth, and runs `Rdot` serially; `control1` is then flat to four decimal places across the whole thread ladder. The gain on small problems reaches $13\times$, and it also removes a source of nondeterminism, since a reduction whose combining order followed thread scheduling was the mechanism behind upstream’s varying iteration counts.

2.4 Timers that reported worker-seconds as elapsed time

The reduced per-formula timers summed time across threads and printed the sum where elapsed time was expected, so a threaded run appeared to spend more time in a region the more it was parallelised. They are now reported as worker-seconds, labelled as such.

3 Reading high-precision input

This defect is not a performance matter and is the one most likely to have silently corrupted someone's results.

QD's decimal-to-dd_real converter overflows internally once the mantissa reaches 309 or more digits, and **returns success while producing NaN**. A 600-digit spelling of an ordinary-magnitude number therefore entered the SDP matrices as NaN and surfaced later as a bogus **cholesky miss condition** at iteration 0 — the solver reporting an infeasible problem when the problem was fine.

The isolation is clean. On a user's 600-digit input, upstream terminates after **0 iterations** with `objValPrimal = -0.0`; this fork reaches `pdOPT` in **43 iterations**. Truncating that same input to **45 significant digits** — beyond double-double's own ~ 32 -digit resolution, and far below the converter's ~ 309 -digit limit — makes *upstream* solve it correctly. Token length is the only thing that changed. Nor is the truncation doing arithmetic work: this fork's answers on the 600-, 45- and 30-digit inputs agree to ten significant digits, and a difference that small cannot turn a NaN into a correct solve.

The reader now validates full decimal syntax, keeps the original conversion path for mantissas up to 308 digits so ordinary inputs stay bit-identical, normalises longer mantissas with the exponent preserved, and checks parser status *and* finiteness rather than letting a NaN reach the solver.

4 Floating-point contraction, and why this fork pins it off

4.1 The mechanism

GCC and clang default to `-ffp-contract=fast` for C++. Double-double arithmetic is header-inline, so *the compiler* decides whether the compensation terms of the double-double algorithms get folded into a fused multiply-add. Folding there is not an identity. The whole point of a `two_sum` is to compute the rounding error that the primary operation discarded; an FMA that keeps the intermediate product exact deletes the very quantity being measured, and the double-double invariant is lost.

The platform dependence follows immediately. On baseline `x86-64` there is no FMA for the compiler to fold into, so those binaries behave as if contraction were off. On `aarch64`, `fmadd` is baseline ISA, so they do not. The two platforms then run different arithmetic on the same source.

4.2 The fix, and its cost

This fork builds with `-ffp-contract=off` **by default**, applied to every SDPA and `mplapack` translation unit *and* to the bundled QD. With it, an M1 build matches the published `x86-64`

reference on **20 of 20** SDPLIB problems — iteration counts and full-precision objectives alike — where the unpinned build matches on none at the state-hash level.

Hardware FMA is not lost. `-ffp-contract=off` does not disable an explicit `__builtin_fma`, so QD’s `two_prod` keeps its single `fmsub` and the `aarch64` speedup of §2.2 is retained in full — verified at the instruction level, not assumed. The cost is $\sim 1\%$ per iteration at one thread and $\sim 1.7\%$ at eight. On `x86-64` the output is unchanged and so is the object code: all 76 translation units compile to byte-identical `.text` with and without the flag.

`--enable-fp-contract=fast` recovers the old, platform-dependent behaviour. Iteration counts published for `arm64` *before* this became the default are counts of the unpinned solver; the per-problem deltas are in `bench/b4_mac_rebaseline.tsv`.

5 What was threaded, and what it costs correctness

Four regions are threaded here that are serial or ineffectively threaded upstream.

5.1 The B constraint loop

The dense-route Schur-complement build. This was the largest single gain in the original port and is where the SDPLIB improvements come from.

5.2 The sparse Schur-complement Cholesky

Bit-identical at any thread count by construction, and this is a property of the loop’s dependence structure rather than of the arithmetic — so it does not depend on the working precision, nor on the values of the work gates. For a fixed pivot, each `k1` owns exactly one destination row; every read comes from the pivot row; and a barrier per pivot preserves pivot order. No sum is reassociated, because no destination is written by two workers.

5.3 The sparse Schur-complement assembly

Parallel over i -groups, serial over blocks. Each worker holds private `work1/work2` scratch matrices, so concurrent groups cannot race through shared scratch, and the accumulation into B is disjoint *within* a block. That last qualification is load-bearing and is the subject of §6.

5.4 Cholesky panel kernels and the X/Z inverse-Cholesky triangulars

Threaded behind measured work gates.

5.5 The triangular solves: forward yes, backward no

The forward substitution is a *scatter*: within a row the writes go to distinct destinations, so there is no reassociation and the threaded result is bit-identical by construction. The sibling `sdpa-gmp-omp` threads it on that basis.

The backward substitution is a *dot product*. Row i ’s off-diagonals all accumulate into the same x_i , so splitting the row across workers reassociates a finite-precision sum — and no arrangement avoids it, because row $i - 1$ genuinely reads x_i , so the forward pass’s defer-the-write trick does not transfer.

The question is therefore not *whether* the answer changes but *which* answer it changes to, and it is worth being exact about what the sibling fork does, because an earlier draft of this document got it wrong. `sdpa-gmp-omp` splits each row into a **fixed number of chunks** (64 by default, *independent of the team size*), accumulates each chunk in a scratch at twice the vector’s precision, and folds the partials in a fixed order. That buys two properties: the result is identical at 1, 8, 32 and 64 threads and across reruns, and each chunk rounds once where the legacy loop rounded at every one of the row’s ~ 800 subtractions.

It does **not** reproduce the legacy left fold bit for bit, and the implementation does not claim it does — its own source says the parallel pass “does change the answer”, which is why it ships opt-in behind `SDPA_SOLVE_BACKWARD` and defaults to serial. Nor is the accuracy claim a theorem: higher-precision pairwise-style accumulation being no worse than a left fold is a design intent to be checked empirically, for every input, rather than proved.

`dd` is not blocked here by a missing widened accumulator. `qd_real` — four doubles, ~ 64 digits, declared in the same headers as `dd_real` — is a direct analogue of GMP’s doubled chunk scratch. What it would buy is the same thing GMP gets: a *new deterministic algorithm with a new answer*, not a way to parallelise the legacy reduction unchanged. There is no finite-width argument by which “enough extra digits” proves equality with a computation that rounded after every subtraction; the only general way to reproduce that left fold exactly is to preserve its dependence order, which is precisely what removes the reduction.

So the choice on `dd` is “accept and validate a new deterministic algorithm, probably opt-in” versus “keep the current answer and keep the reduction serial”. This fork currently takes the second, and the reason is the contract stated in §8.1: results here are bit-identical to the serial computation, and changing the backward pass would spend that for a fraction of one phase.

6 A data race in the threaded assembly, and its fix

6.1 What shipped

The threaded sparse assembly’s worksharing loop carried `nowait`, justified by the observation that the B writes are `+=` into distinct slots *within a block*. That observation is true and insufficient. `nowait` lets one worker enter block $l + 1$ while another is still in block l , and *across* blocks the slots are not distinct: a constraint pair that is nonzero in two blocks accumulates into the same entry from both.

The consequence, on SDPLIB `truss6` at default settings, where the correct objective is `-9.0100140477088098e+02`. Five consecutive runs of `v7.1.3-omp.2` returned

2.1611901436772943e+06	9.7205414006274572e+04	-9.0093934416427931e+02
1.6632246196208220e+04	9.8204796932176279e+05	

The third value is the instructive one: it is wrong in the *fifth significant digit*, not absurd. A tolerance check against a reference would have accepted it. On one host, three of five wrong values agreed with each other to four digits, so a small-sample spread test would have called the run stable.

6.2 Why nothing caught it

Three reasons, none of them “we did not test”.

1. **The CI fixture was block-diagonal by construction.** No Schur entry is ever written twice in it, so the race had nothing to land on. Every stream-identity assertion ran on that fixture, and every one of them passed.
2. **The large problems were not immune, and were believed to be.** Both dE3 and dE4 report `blocks_disjoint=0`. The race fired on dE4 *once in seventeen* threaded runs. They escaped notice because a rare race compared *once* is a race not tested for.
3. **The guarantee was stated in a form that misdirected every check.** It was always written as “bit-identical at any thread count”, which points every test at comparisons *between* thread counts. A race that moves both sides passes those comparisons. The check that catches it is repetition at *one* thread count, and nobody had written it.

6.3 The fix

The barrier is taken, but only where the finished aggregate map *proves* it is needed. One setup pass asks whether any Schur entry is written by more than one SDP block:

```
if (blocks_disjoint) → #pragma omp for schedule(dynamic) nowait
else → #pragma omp for schedule(dynamic)
```

with a single extracted `Newton::assemble_group()` body shared by both worksharing forms, so the two paths cannot drift apart. Block-diagonal problems keep `nowait`; cross-block problems take the barrier. *The fast path is taken because the map was checked* — which is precisely the discipline whose absence caused the defect.

The disjointness pass itself uses two bitsets over the stored Schur entries, `seen_any` and `seen_blk`, with `seen_blk` cleared by re-walking the block’s own locations rather than by maintaining a scratch list. That is $O(\text{pairs})$ and two bits per stored entry, replacing an earlier one-int-per-entry map that was $16\times$ larger.

Cost, on the four-iteration budget protocol: dE4 +0.7%, dE3 via the fill route +2.2%, and nothing measurable in memory — dE4 is 382.0 MB against 382.2 MB before the fix.

6.4 The regression fixture, and why its first version was worthless

`tests/gen_crossblock_fixture.py` generates an $m=480$ problem that routes sparse, threads, and shares Schur entries between blocks. Its first version used a block stride of 7, and the *racing* binary passed it 6/6: blocks seven apart are never in flight together. Adjacency is load-bearing, and the stride must be 1.

CI now asserts three things: that repeated runs at *one* thread count agree; that the threaded assembly stream equals the serial one; and that the overlap policy is the computed one *in both directions*, so a fixture that stopped exercising the cross-block case would fail rather than pass vacuously.

7 The factorisation choice

7.1 Four gates

The chooser decides whether B is stored and factored densely or sparsely. It is a cascade of four gates; a problem goes dense as soon as any gate rejects.

1. a coarse structural screen, which decides the great majority of problems;
2. a screen on the number of blocks contributing, against $c \cdot m$;
3. a pre-screen on the **aggregate** sparsity pattern of B , against $F \cdot m^2$;
4. the real test, on the **ordered fill** of the symbolic Cholesky factor, against $F \cdot m^2$.

Write ρ_{agg} for the density of the aggregate pattern and ρ_{fill} for the density of the ordered fill, both as fractions of m^2 .

7.2 One constant doing two unrelated jobs

Gates 2 and 3 were both driven by a single constant, so neither could be retuned without moving the other. Splitting it frees gate 3 — and once gate 3 is free, it stops being a tuned threshold at all.

7.3 Gate 3 is not a heuristic

Lemma 1. *Symbolic Cholesky factorisation only adds entries: every nonzero of the aggregate pattern is a nonzero of the factor’s pattern. Hence $\rho_{\text{fill}} \geq \rho_{\text{agg}}$ always.*

Corollary 1. $\rho_{\text{agg}} > F \implies \rho_{\text{fill}} > F$. So whenever gate 3 rejects at cutoff F , gate 4 would also have rejected at cutoff F .

Gate 3 at F is therefore a *provable early exit* from gate 4, not an independent heuristic: it can only skip work, never change the verdict. And that makes F the **largest sound cutoff** for gate 3. Anything smaller rejects problems gate 4 would have accepted.

7.4 What the old value was, and why it was wrong

Gate 3 rejected at $0.25 \cdot m^2$ while gate 4 rejected at $0.40 \cdot m^2$. Gate 3 was thus the *stricter* test, and every problem with $\rho_{\text{agg}} \in (0.25, 0.40]$ was sent dense **without gate 4 ever running**. The cheap pre-screen overruled the real test.

The correction is to demote gate 3 to gate 4’s own constant. **One tunable, $F = 0.40$, unchanged, now replaces two.** It is worth being precise about what this is not: 0.25 was not a value tuned on evidence that later went stale. It was conservatism with nothing behind it, and the soundness argument above shows there was never a regime in which it could pay.

7.5 Who this affects

Route decisions depend only on sparsity structure, so a census transfers across precisions and across the sibling forks.

SDPLIB: nobody. Of 92 problems, **none** change route. 84 are decided at gate 1, 6 at gate 2, and the single problem that reaches gate 3 — **truss5** — has $\rho_{\text{agg}} = \rho_{\text{fill}} = 1.0$ and goes dense under both rules.

Bootstrap problems: most of them. On a 221-instance census, **167 instances across seven structural classes** change dense $\rightarrow$ sparse. Every one of those 167 has $\rho_{\text{agg}} \in [0.257, 0.362]$ and $\rho_{\text{fill}} \in [0.259, 0.367]$: above the old 0.25, and **not one above 0.40**. Gate 4 said sparse on all of them.

7.6 Measured effect

One representative of *each* of the seven structural classes was timed — seven timed inputs, one per class, not 167. The classes are complete coverage of the switch set; the timings are seven observations.

m	legacy (dense)	fill (sparse)	faster	less memory
2,439	1.92 s / 116 MB	0.61 s / 50 MB	3.17×	2.30×
4,489	9.22 s / 392 MB	3.59 s / 213 MB	2.57×	1.84×
5,278	14.50 s / 519 MB	3.88 s / 234 MB	3.74×	2.22×
6,067	22.97 s / 671 MB	4.50 s / 277 MB	5.11×	2.42×
8,359	66.20 s / 1,289 MB	15.13 s / 587 MB	4.38×	2.20×
10,614	150.47 s / 2,032 MB	22.36 s / 831 MB	6.73×	2.45×
11,227	175.81 s / 2,358 MB	47.94 s / 1,183 MB	3.67×	1.99×

Raw rows: `bench/dd-port3-2026-08-24/dd_fill_seven_structures.tsv`.

7.7 What this evidence does not establish, and where the rest of it comes from

The table above is a **fixed four-iteration, per-iteration** experiment. *No time-to-solution comparison was performed on `dd`*, and none can be: these instances do not converge at double-double precision under either tolerance tested (the shipped `epsilonStar = 1e-30` and a relaxed `1e-20`). A looser parameter choice is a different experiment and nothing here rules it out.

The other half of the case comes from the sibling fork. `sdpa-gmp-omp` promoted the same policy on 2026-08-18 after five review rounds, on the *same seven structures*, measured **at full convergence** ($m=6067$: dense 2708 s against sparse 310 s), on two architectures, at 256 and 512 bits, at nine labelled thread points, with no reversal anywhere — and at *one* thread, where sparse still won 2.60–3.06×, which shows the 0.25 was already mis-routing in the serial regime it was calibrated for. Where the dense arm converged, phases and iteration counts were equal per pair and the solution vectors agreed to 6.3e−38.

7.8 Not “sparse always wins”

`truss5` has an ordered fill of 1.0 — a completely dense factor — and dense is genuinely about 1.4× faster there. Both the old and the new rule route it dense. That is the point of testing the fill rather than assuming an answer.

7.9 What the change costs, stated plainly

`SDPA_BMAT_MODE` is **result-changing by design**. Dense and sparse are different factorisation routes and follow different iterate trajectories, so results can differ in the last digits. Across a 48-cell harness, 30 cells differ between routes, while every cell that *converges* agrees on the

objective. Anyone needing bit-reproducibility against an earlier run must keep the mode fixed; `SDPA_BMAT_MODE=legacy` reproduces the pre-2026-08-24 chooser exactly.

8 Reproducibility: established, not asserted

8.1 What is guaranteed

- The assembled Schur complement is **bit-identical** at any thread count, and across repeated runs at one thread count.
- The sparse Cholesky factor is **bit-identical** at any thread count, by the dependence argument of §5.2.
- A checksum over every scalar of the final `xMat/yVec/zMat` is identical at 1/2/4/8/24 threads.
- Results are identical between `arm64` and `x86-64`, given the contraction pin of §4.

Unpatched upstream has none of these. Threaded, its iteration counts and even its objectives vary run to run: `control1` ran 69/74/88 iterations across three repeats at 24 threads on one host and 69/82/109 at 8 threads on another, where this fork ran 71/71/71 on both.

8.2 Why the printed solution is the wrong thing to compare

A `dd_real` carries about 32 significant digits; the printed result fields carry 17. A factor can therefore change without any printed field moving. Every reproducibility claim here is made against the *structures* — the assembled matrix, the finished factor — and not against printed digits.

8.3 The oracles

variable	what it produces
<code>SDPA_SPCHOL_DIGEST</code>	fingerprint of the finished factor: records, bytes, 64-bit FNV-1a
<code>SDPA_SPCHOL_DIGEST_DUMP=file</code>	the factor's canonical stream, appended
<code>SDPA_BMAT_ASM_DIGEST</code>	the same fingerprint, for the assembled Schur complement
<code>SDPA_BMAT_ASM_DUMP=file</code>	the assembly's canonical stream

The dumps are the proof-grade comparison: run twice, `cmp` the files. A fingerprint is strong evidence, not proof, and is described that way wherever it is printed. Both streams carry a length frame before every variable-length field, so two different structures cannot serialise to the same bytes, and the two streams use different tags so an assembly stream can never compare equal to a factor stream.

One operational caveat that cost a full sweep: the dump files are opened in append mode ("`ab`"), so a comparison harness must `rm -f` them before each run or every problem will appear to race.

8.4 Negative controls, and why they are not optional

A comparison that cannot fail is worse than no comparison, because it reports success. Each oracle therefore has a hook that injects the smallest change it must detect, and CI asserts that the hook actually changes what it claims to change. These require `-DSDPA_SPCHOL_TEST_HOOKS` at build time; a release build *refuses* them rather than ignoring them, because a knob that silently does nothing misleads whoever set it.

hook	what it injects
<code>SDPA_SPCHOL_MUTATE=1</code>	one ulp into the finished factor
<code>SDPA_BMAT_ASM_MUTATE=1</code>	one ulp into the assembled matrix
<code>SDPA_BMAT_TEST_BREAK_INVARIANT=1</code>	a violation of the $\rho_{\text{fill}} \geq \rho_{\text{agg}}$ invariant
<code>SDPA_BMAT_TEST_F2_STALE=1</code>	the worst-case stale- G read a since-fixed defect could produce
<code>SDPA_SPCHOL_TEAM_OVERRIDE=n</code>	asks the runtime for n threads, leaving the gates' team alone
<code>SDPA_SPCHOL_FAIL_AT=i</code>	declares pivot i indefinite

The last two are a different idea from the first four: not negative controls for an oracle, but the only way to execute two branches that no input reaches on purpose. `TEAM_OVERRIDE=1` is the only route into the “requested a team and received one thread” fallback — the dispatcher’s own `team < 2` test exits earlier and by a different path, so without the hook that branch is dead to every test — and CI additionally requires the fallback’s factor to be byte-identical to the threaded one. `FAIL_AT` makes the failure return from a *partly-threaded* factorisation executable, and CI requires that threaded pivots ran first, or the test would prove nothing about that path. When it fires, the diagnostic says the failure was **declared**, not measured.

8.5 Route fixtures

`tests/gen_route_fixture.py` generates eight cases straddling the gates and asserts the exact route taken under `auto`, `fill` and with the variable unset. One case, `wide`, is policy-divergent, and CI fails if it stops diverging — because the assertion that unset behaves like `auto` is vacuous unless at least one case can tell the policies apart.

9 Environment variables

Two rules hold for every variable below, and are tested in CI rather than merely stated.

1. **A typo is refused, never treated as the default.** `SDPA_SPCHOL_MODE=Auto` or `SDPA_DD_MIN_SPCHOL_WORK=-5` stops the run with a diagnostic. Silently falling back would let a mistyped variable select the very path the caller was trying to avoid — and `-5` in particular used to be *accepted*, wrapping to 18446744073709551611, i.e. “never parallel” while looking configured.
2. **Validation happens on every route.** A knob is parsed once per process at a point both the dense and the sparse route reach, so a dense-route problem refuses a malformed value it would never otherwise consult. This fork has been bitten five times by checks placed where only one route could reach them.

9.1 Choosing how the Schur complement is stored and factored

variable	values	changes results?
SDPA_BMAT_MODE	auto (default) · fill (synonym) · yes legacy · dense · sparse	
SDPA_BMAT_MAX_GB	positive number of GB	no
SDPA_BMAT_LOG	1 on, 0 off	no

auto is the current default policy, which since **2026-08-24** is the fill-derived rule of §7; **fill** is retained as an explicit synonym so opt-in-era scripts keep working; **legacy** restores the pre-2026-08-24 chooser exactly. **dense** and **sparse** force a route. All values are strictly parsed. **SDPA_BMAT_LOG=1** prints which gate decided and why.

SDPA_BMAT_MAX_GB caps the dense allocation and is enforced on *every* route to dense, so a problem that would silently need more memory than the machine has fails with a number instead. It fails **closed**: an empty value is an error rather than “no cap”, because **SDPA_BMAT_MAX_GB="\$TYPO"** is the ordinary way an empty value reaches a job script, and treating it as no-cap would fail *open* on a safety limit.

Available memory is reported, never acted on. With **SDPA_BMAT_LOG=1**, and in the message a **SDPA_BMAT_MAX_GB** refusal prints, the dense requirement is shown next to what the machine has free, so “needs 21 GB” can be read against something. Nothing branches on it, and CI asserts that nothing does: a route chosen from free memory would be a route that changes with the machine, which is the opposite of what this fork guarantees. The figure is Linux-only (MemAvailable from `/proc/meminfo`; elsewhere the line says it is not readable rather than guessing), and it reports the *host*, so inside a memory cgroup it is an upper bound.

9.2 Threading the sparse Cholesky

variable	values	changes results?
SDPA_SPCHOL_MODE	auto (default) · serial · force · no legacy	
SDPA_DD_MIN_SPCHOL_WORK	integer, default 10000	no
SDPA_DD_MIN_SPCHOL_WIDTH	integer, default 8	no
SDPA_DD_MIN_SPCHOL_TOTAL	integer, default 0 (pass-through)	no
SDPA_SPCHOL_LOG	1 on, 0 off	no

legacy runs an independent transcription of the pre-refactor expression. It exists as an **oracle**: **serial** and the threaded path share their inner kernel, so comparing those two cannot detect a bug in the shared kernel, whereas **legacy** can.

force forces or fails — in a build without OpenMP it is refused rather than silently running serial, so a benchmark cannot believe it measured the threaded path when it measured the other one.

The gates were calibrated on **dd**, on two architectures (i9-13900K/24 cores and Apple M1 Max/10 cores). They are runtime-overridable precisely so a machine that disagrees can be accommodated without a rebuild.

9.3 Threading the sparse B assembly

variable	values	changes results?
SDPA_BMAT_ASM_MODE	auto (default) · serial · parallel	no
SDPA_BMAT_ASM_MIN_PAIRS	integer, default 8000	no
SDPA_BMAT_ASM_SCRATCH_MB	integer MB, default 4096	no
SDPA_BMAT_ASM_LOG	1	no
SDPA_BMAT_ASM_PROFILE	1	no
SDPA_BMAT_ASM_CENSUS	1	no

SDPA_BMAT_ASM_SCRATCH_MB is a **production safety control**, not observability: each worker beyond the first needs two dense matrices of the largest block's order. If the budget admits fewer workers than requested, the team shrinks; it never silently exceeds the budget.

SDPA_BMAT_ASM_MIN_PAIRS = 8000 was calibrated on two architectures. Below roughly 5,000 pairs threading loses — the fork/join costs more than the work — and above roughly 15,000 it wins by 8–10×. The gate is a **proxy**: pair count does not fully determine the work, since a group's cost also scales with its block dimension, so a problem near the boundary may benefit from overriding it. A work-based replacement was attempted and the result was *negative*: no static pair-count threshold is safe on both calibration machines. That is recorded here rather than quietly dropped.

9.4 Profiling and census

SDPA_BMAT_ASM_PROFILE=1 splits the **Make bMat** phase into zeroing, SDP assembly and LP assembly. SDPA_SOLVE_PROFILE=1 splits the triangular solve into its forward and backward passes. Both are reported in the phase table and cost a few clock reads per iteration when off.

SDPA_BMAT_ASM_CENSUS=1 reports, per SDP block: pair count, group count, largest group, the formula mix, and how many pairs consume the F2/dense- A_j path. Useful for judging whether a problem can balance across threads before trying.

10 Exit status

Upstream exits 0 on *every* path, including fatal errors, so a crashed run is indistinguishable from a solved one in any harness that checks exit codes. This fork distinguishes them.

outcome	exit
solver ran to any stopping condition (pdOPT, pdFEAS, pFEAS, dFEAS, pdINF, pINF_dFEAS, pFEAS_dINF, pUNBD, dUNBD, noINFO)	0
iteration limit reached	0
infeasibility / unboundedness detected	0
malformed input, unreadable file, invalid parameter	1 , with a diagnostic (line-numbered for data files)
numerical failure with nothing valid to print — no iteration completed, or the updated X/Z left the cone <i>and the rollback could not be refactored</i>	2 , solveStatus = FAILURE, no solution section
recoverable late failure after k good iterations — a Schur factorisation failure, or an X/Z update that left the cone and <i>was</i> rolled back; the last valid iterate is printed and labelled	3 , solveStatus = PARTIAL, failureIteration = k

Infeasibility and the iteration limit are valid mathematical results, not errors, and are reported as such.

11 What is not established

Stated positively so that nobody has to infer it from silence.

- **No time-to-solution comparison exists for the large sparse problems.** dE3 and dE4 do not converge at double-double precision under either tolerance tested ($1\text{e-}30$ shipped, $1\text{e-}20$ relaxed). Every figure quoted for them is *per-iteration cost over a fixed four-iteration budget*. A looser parameter choice is a different experiment.
- **The route census is structural, the timings are seven observations.** Seven representatives cover all seven classes containing the 167 switch instances; 167 instances were not timed.
- **The assembly work gate has no principled replacement yet.** The attempt to derive one returned a negative result on two machines.
- **The OpenMP work thresholds carry a precision caveat.** They were calibrated at double-double. The sibling gmp fork inherits the mechanism but would need them re-measured at 256 bits.
- **Short cells are quoted to two significant figures on purpose.** The 24-thread dE4 cell spreads 8.2% across nine runs in three campaigns (6.234–6.761 s), and the 24-thread dE3 sparse cell spreads 12% across six runs. Ratios derived from them are “about $7.2\times$ ” and “about $5\times$ ”; every earlier revision that quoted three figures had to move the number at the next re-measurement.
- **Threaded upstream is nondeterministic**, so fork-versus-upstream figures are observed end-to-end speedups rather than strictly same-trajectory comparisons.

12 Sources

- Upstream: <https://github.com/nakatamaho/sdpa-dd> at 6eaad8d9. The base commit is not an ancestor of this repository’s history; fetch it explicitly to diff.
- This fork: <https://github.com/Canonical111/sdpa-dd-omp>
- Sibling fork, arbitrary precision, where the factorisation policy was derived and measured at full convergence: <https://github.com/Canonical111/sdpa-gmp-omp>
- Benchmark tables and methodology: `BENCHMARKS.md`; raw per-repeat rows and per-file provenance under `bench/`, with `bench/validate.py` as an integrity check.
- Build and verification: `INSTALL.md`.

Every modified or added source file carries an in-file, dated change notice naming its own licence: GPLv2 §2a for the SDPA sources, and the two-clause BSD terms for `mplapack/`, which

carries no GNU licence at all. The complete, always-current list of changes is a diff against upstream; an enumeration went stale twice and is deliberately not repeated here. `git log` carries the rationale per change.